SYSTEM ARCHITECTURE DIAGRAM

Project Title	AI Academic Mentor
Team	CSE4104-7D-T06
Section	7D
Institution	Northern University of Business & Technology Khulna (NUBTK)
Course	CSE 4104 — Software Development III
Document	System Architecture Diagram
Date	16 November 2026
Github Repository Link	https://github.com/Santu-Mondal/AI-Academic-Mentor-Personalized-Learning-Study-Assistant/tree/411142fcd5c145a92f334e7a9af446e864ef6c69/documentation
Instructor	Md. Riaz Mahmud, Assistant Professor, CSE — NUBTK

Team Information

SL No.	Member Name	Student ID	Role
1	Efaz Rahman	11230121162	Team Leader
2	Santu Mondal	11230321312	Frontend Developer
3	MD Ruhan Tanveer	11230321335	Backend Developer AI Integration Lead

1. Initial System Architecture Diagram

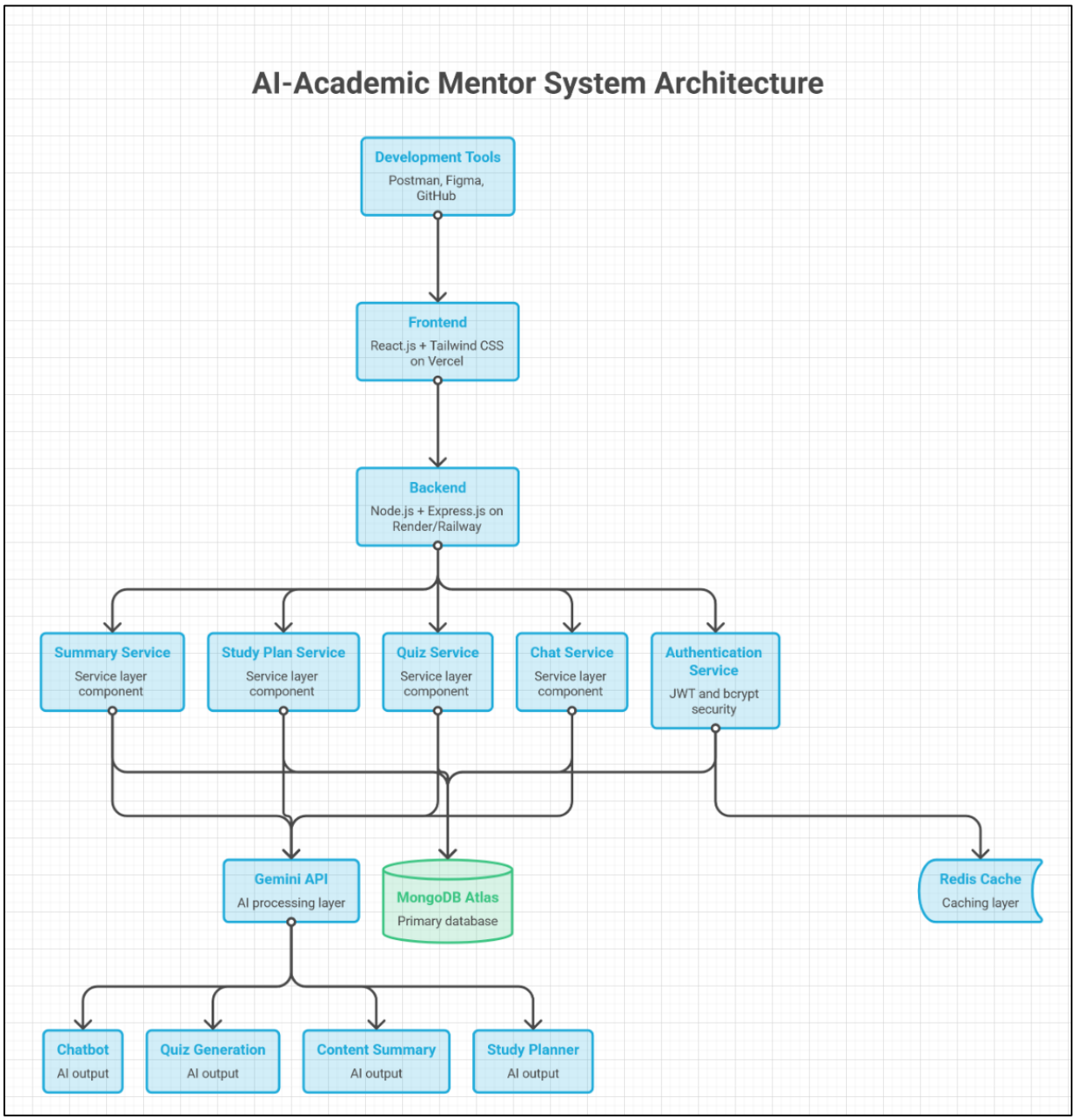


Fig — Initial AI Academic Mentor System Architecture Diagram

2. Layer-by-Layer Breakdown

The architecture is organized into six hierarchical layers. Each layer encapsulates a distinct set of responsibilities and communicates with adjacent layers via defined interfaces.

Layer	Component	Technology / Role
1 — Dev Tools	Development Toolchain	Postman · Figma · GitHub — build & test environment
2 — Frontend	React.js + Tailwind CSS	SPA deployed on Vercel; user-facing interface layer
3 — Backend	Node.js + Express.js	REST API server on Render; central routing hub
4 — Services	Summary Service	AI-powered content summarization module
	Study Plan Service	Personalized AI study schedule generator
	Quiz Service	Dynamic quiz creation and evaluation engine
	Chat Service	Multi-turn conversational AI session handler
	Authentication Service	JWT token issuance + bcrypt password hashing
5 — Data & AI	Gemini API	Google Gemini 2.0 Flash — AI processing engine
	MongoDB Atlas	Primary database — users, sessions, quizzes, plans
	Redis Cache	Caching layer for sessions and rate-limit state
6 — AI Outputs	Chatbot	Natural-language academic mentor responses
	Quiz Generation	On-demand subject-specific question sets
	Content Summary	Intelligent note and document summarization
	Study Planner	AI-generated structured weekly study schedules

3. Component Descriptions

3.1 Development Tools

Postman is used for REST API testing and endpoint validation. Figma provides the UI/UX design workspace from which the React implementation is derived. GitHub hosts the version-controlled source repository for the entire project codebase.

3.2 Frontend

The client layer is built with React.js and Tailwind CSS, deployed on Vercel as a static single-page application. It communicates with the backend over HTTPS via the Fetch API. The design system uses Cormorant Garamond and DM Sans typography with a deep navy, cobalt, gold, and teal color palette.

3.3 Backend

The application server is an Express.js application running on Node.js 18+, deployed on Render. It exposes a RESTful API, manages route-level authentication middleware, enforces rate limiting via express-rate-limit, and applies HTTP security headers via Helmet.

3.4 Service Layer

Five service modules sit within the backend, each responsible for a discrete functional domain:

- Summary Service — orchestrates AI-powered note summarization requests.
- Study Plan Service — generates and persists personalized study schedules.
- Quiz Service — handles quiz generation, submission, and scoring.
- Chat Service — manages multi-turn conversation sessions and history.
- Authentication Service — issues JWT tokens and enforces bcrypt hashing (cost factor 12).

3.5 Data & AI Infrastructure

Gemini API (Google Gemini 2.0 Flash) serves as the AI inference engine, receiving structured prompts from all four feature services and returning streamed completions. MongoDB Atlas is the primary persistent datastore, holding user accounts, session records, quiz results, and study plans. Redis Cache provides a fast in-memory layer for session tokens and rate-limit counters.

3.6 AI Output Modules

The four terminal outputs produced by the Gemini API integration are: Chatbot (natural language conversational responses), Quiz Generation (structured MCQ and True/False question sets), Content Summary (condensed academic note summaries), and Study Planner (weekly schedule objects rendered as exportable plans).

4. Data Flow Summary

A user action in the browser triggers an authenticated HTTPS request from the React frontend to the Express backend. The backend validates the JWT, applies rate-limiting, and dispatches the request to the appropriate service module. Feature services construct prompt payloads and forward them to the Gemini API; responses are streamed back to the client. Concurrently, persistence operations are executed against MongoDB Atlas, and session/token state is read and written via Redis Cache.

5. Architectural Notes & Observations

The following observations have been identified for future diagram revision and architectural refinement:

#	Observation / Note
1	Redis is connected only via the Authentication Service in the diagram. In production, Redis should also cache Gemini responses and enforce rate-limiting across all service routes.
2	The 'Development Tools → Frontend' arrow is conceptually atypical for a runtime architecture diagram. Dev tooling should be represented in a separate 'Dev Environment' swim lane.
3	The Announcement / Notification system added in Week 4 is not yet reflected in this diagram and should be incorporated in the next revision.
4	The deployment target is listed as 'Render / Railway'. The current confirmed deployment environment is Render exclusively; this should be corrected.